

ESP Thread Border Router

Module Gateway H2 supports running the ESP Thread Border Router SDK alongside ESP32 series Wi-Fi SoCs. This SDK is built based on ESP-IDF and OpenThread, running the Thread network on the H2 while the H2 communicates with the main processor via a serial interface.

1. Preparations

1. **Environment Setup:**

Refer to the [ESP-IDF - Getting Started with ESP32S3](#) tutorial to complete the basic compilation environment.

ESP-IDF version

It is recommended to use ESP-IDF version **v5.3.1** for compiling this example.

```
git clone -b v5.3.1 --recursive https://github.com/espressif/esp-idf.git
cd esp-idf
./install.sh
. ./export.sh
```

2. Hardware Products Used:

- [ESP32 Downloader](#)
- [Module Gateway H2](#)
- [CoreS3](#)



- 3. The subsequent tutorial uses `idf.py` commands which depend on ESP-IDF. Before running the commands, execute `./export.sh` in ESP-IDF to activate the related environment variables. For detailed instructions, please refer to the [ESP-IDF - Getting Started with ESP32S3 Tutorial](#).

2. Compiling the RCP Firmware

- 1. Before compiling the Thread Boarder Router firmware, you need to generate the RCP firmware first. Follow the instructions below to navigate to the corresponding RCP firmware directory and set the build target to **esp32h2**.

```
cd $IDF_PATH/examples/openthread/ot_rcp
idf.py set-target esp32h2
idf.py menuconfig
```

- 2. Use `idf.py menuconfig` to enter the configuration menu. In `menuconfig`, configure:
Component config -> OpenThread RCP Example - Enable **OPENTHREAD_NCP_VENDOR_HOOK**
- 3. After finishing the configuration, execute the following command to build the RCP firmware.

```
idf.py build
```

3. Compiling the ESP Thread BR Firmware

- 1. Clone the project.

```
git clone https://github.com/Ocean-lhy/esp-thread-br.git
```

- 2. Switch to the corresponding branch based on the main controller you are using.

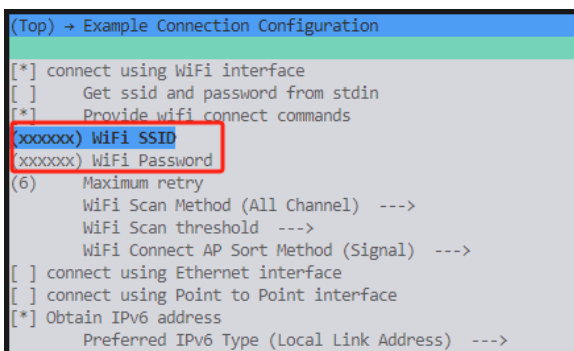
```
# For CoreS3
git checkout demo_for_core_s3_test
cd examples/thread_border_router_credential_sharing
idf.py set-target esp32s3

# For Core2 v1.0 and v1.1 (the power management chips are AXP192 and AXP2101 respectively; configuratio
git checkout demo_for_core_2_test
cd examples/thread_border_router_credential_sharing
idf.py set-target esp32

# For Core
git checkout demo_for_core_test
cd examples/thread_border_router_credential_sharing
idf.py set-target esp32
```

- 3. Use `idf.py menuconfig` to enter the configuration menu. In `menuconfig`, configure the Wi-Fi settings:
Component config -> Example Connection Configuration

```
idf.py menuconfig
```



```
(Top) -> Example Connection Configuration
[*] connect using WiFi interface
[ ]   Get ssid and password from stdin
[*]   Provide wifi connect commands
(xxxxxx) WiFi SSID
(xxxxxx) WiFi Password
(6)   Maximum retry
      WiFi Scan Method (All Channel) --->
      WiFi Scan threshold --->
      WiFi Connect AP Sort Method (Signal) --->
[ ] connect using Ethernet interface
[ ] connect using Point to Point interface
[*] Obtain IPv6 address
      Preferred IPv6 Type (Local Link Address) --->
```

- 4. Compile and flash the ESP Thread BR firmware.

```
idf.py build
idf.py erase_flash
idf.py flash
```

4. Getting Started

- 1.After flashing, restart the device. The device will enter the RCP update state. Once the RCP update is successful, the device will restart and connect to both Wi-Fi and the Thread network. Upon completion of device initialization, the following information will be displayed:
 - A **generate epskc** button
 - A **factoryreset** button
 - The **Border router web server URL**



- 2.Click the **generate epskc** button. The device will generate an epskc and display it on the screen, which can be used for quick network commissioning.



- 3.Within your LAN, use a browser to access the Border router web server URL to view Thread network information.

Thread Network Status

The status of the ESP-OpenThread network, including its IPv6, network, OpenThread, RCP, WPAN, and other components, would be displayed here.

Overview IPv6 Network OpenThread RCP WPAN

[IP Address]

Link Local Address
fe80:0:0:40fa:5c7c:34a5:872c

Routing Local Address
fd00:db8:a0:0:0:ff:fe00:5400

Mesh Local Address
fd00:db8:a0:0:1ee5:b93e:8954:236f

Mesh Local Prefix
fd00:db8:a0:0::/64

[Network Information]

Name
OpenThread-ESP

PANID
0x1234

Partition ID
211565874

Extended PANID
dead00beef00cafe

Border Agent ID
5d8a3792f05393c8c75748dc352d5e84

[OpenThread Parameter]

Version
openthread-esp32/7a305c0284-005c5cefc;
esp32; 2024-12-12 01:57:50 UTC

Version API
458

Role
leader

PSKc
104810e2315100afd6bc9215a6bfac53

[RCP Information]

Channel
21

IEEE EUI-64
744dbdffe60df08

TxPower
20 dBm

Version
openthread-
esp32/7a305c0284-005c5cefc;
esp32h2; 2024-12-12 01:41:58
UTC

[WPAN Status]

Service
associated

- ## 5. Testing

1. Enter `networkkey`, `panid`, and `channel` in the background of the Thread Boarder Router to obtain the network key, panid, and channel of the Thread network.
2. Enter the configuration commands in the SimpleCLI routine and start the Thread network.

3. Enter **state** in the SimpleCLI serial port interaction to view the state of the Thread network. If it is connected to the network as a child/router, the Thread network connection is successful. If it forms a network as a leader, there may be an error in the configuration.
4. Enter **parent** in the SimpleCL serial port interaction to view the parent node of the Thread network; enter **extaddr** to view the extended address of this node.
5. Enter **extaddr** in the serial port interaction of the Thread Boarder Router to view the extended address of this node, which should be the same as the **parent extaddr** in the SimpleCLI routine.
6. Enter **neighbor table** in the serial port interaction of the Thread Boarder Router to view the neighbor nodes of the Thread network, which should include the node of the SimpleCLI routine.

4/4 | Update Time: 2026-04-28